Technical Report: Predicting Firm Fast Growth

Authors: Konstantinos Evagorou, Fazile Brahimi
Institution: Central European University, Data Analysis 3

1. Data Preparation

1.1 Loading and Filtering the Dataset

```
import pandas as pd
import numpy as np

# Load Hungarian firm panel data covering 2010-2015
data = pd.read_csv("cs_bisnode_panel.csv")
data = data.query("year >= 2010 & year <= 2015")

# Focus on 2012 as base year, keeping only active firms with positive sales
data_2012 = data.query("year == 2012 & status_alive == 1 & sales > 0")
data_2012 = data_2012.query("sales / 1000000 <= 10 & sales / 1000000 >= 0.001")

# This gives us 21,723 small-to-medium firms with revenue between €1K-€10M
```

We chose 2012 as the base year because it allows us to look forward one year to 2013 for growth outcomes. The revenue filter ensures we're looking at SMEs that have meaningful operations but aren't large corporations—these are the firms where identifying growth signals matters most for investors.

1.2 Defining Fast Growth

```
# Get 2013 sales for the same firms
data_2013 = data.query("year == 2013 & status_alive == 1")[["comp_id", "sales"]]
data_2012 = data_2012.merge(data_2013, on="comp_id", suffixes=("", "_2013"))

# Calculate growth rate and define fast growth as exceeding 25%
data_2012["growth_rate"] = (data_2012["sales_2013"] - data_2012["sales"]) / data_2012["sales"]
data_2012["fast_growth"] = (data_2012["growth_rate"] > 0.25).astype(int)

# Result: 5,480 firms (25.2%) achieved fast growth
```

The 25% growth threshold isn't arbitrary—it substantially exceeds typical economic growth rates and indicates genuine business expansion rather than just keeping pace with the economy. In our sample, about one in four firms achieves this level of growth, making it ambitious but attainable.

1.3 Engineering Predictive Features

```
# Transform sales to log scale (deals with right skew)
data_2012["sales_mil_log"] = np.log(data_2012["sales"] / 1000000)

# Create age variables - both linear and quadratic
data_2012["age"] = data_2012["year"] - data_2012["founded_year"]
data_2012["age2"] = data_2012["age"] ** 2
data_2012["new"] = (data_2012["age"] <= 1).astype(int)

# Flag firms with majority foreign ownership
data_2012["foreign_management"] = (data_2012["foreign"] >= 0.5).astype(int)

# Convert categorical variables
data_2012["gender_m"] = data_2012["gender"].astype("category")
data_2012["m_region_loc"] = data_2012["region_m"].astype("category")

# Simplify industry codes into broader categories
data_2012["ind2_cat"] = data_2012["ind2"].copy()
data_2012["ind2_cat"] = np.where(data_2012["ind2"] > 56, 60, data_2012["ind2_cat"])
data_2012["ind2_cat"] = np.where(data_2012["ind2"] < 26, 20, data_2012["ind2_cat"])
data_2012["ind2_cat"] = np.where((data_2012["ind2"] < 55) & (data_2012["ind2"] > 35), 40, data_2012["ind2_cat"])

# One-hot encode categorical features: 8 original features → 15 model inputs
X = pd.get_dummies(data_model[features], drop_first=True)
```

The quadratic age term is important because firm growth doesn't follow a simple linear pattern. Very young firms may grow rapidly but erratically, middle-aged firms often hit their stride, and older firms may plateau. The squared term lets the model capture this lifecycle.

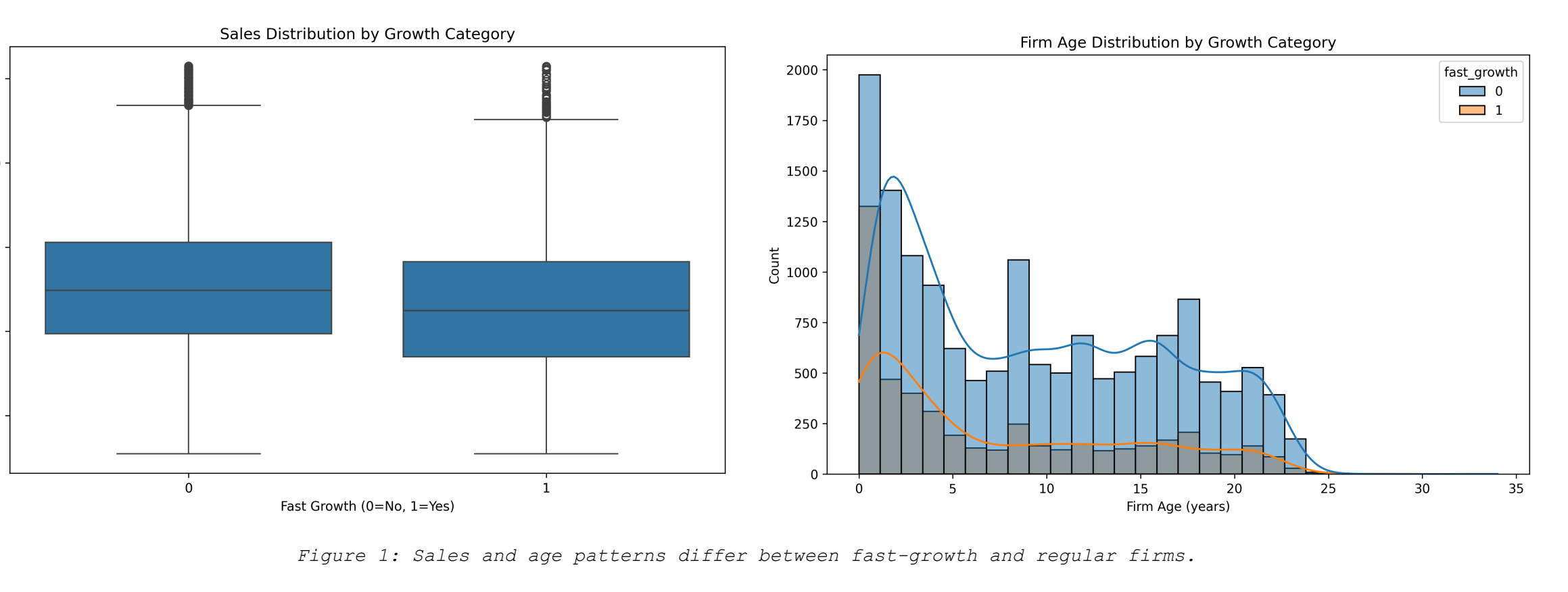

Figure 1: Sales and age patterns differ between fast-growth and regular firms.

2. Model Selection and Training

2.1 Logistic Regression (Baseline)

```
from sklearn.linear_model import LogisticRegressionCV

logit = LogisticRegressionCV(cv=5, random_state=42, max_iter=1000)
logit.fit(X, y)
```

We started with logistic regression as a baseline. It's simple, interpretable, and gives us coefficients we can explain to business stakeholders. The CV version automatically tunes the regularization strength through 5-fold cross-validation.

2.2 Random Forest

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=100,
    max_depth=None,
    min_samples_split=2,
    class_weight='balanced',
    random_state=42
)
rf.fit(X, y)
```

Random forest builds 100 decision trees, each trained on a bootstrapped sample of the data. This ensemble approach handles non-linear relationships well and is robust to outliers. The balanced class weights help since we have somewhat imbalanced classes (25% vs 75%).

2.3 Gradient Boosting (Final Choice)

```
from sklearn.ensemble import GradientBoostingClassifier

gb = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)
gb.fit(X, y)
```

Gradient boosting builds trees sequentially, with each new tree focusing on the mistakes of previous trees. We keep the trees shallow (max_depth=3) to avoid overfitting—this means each tree makes simple decisions but together they form a powerful predictor. The learning rate of 0.1 means each tree contributes moderately to the final prediction.

3. Cross-Validation Results

3.1 How We Evaluated Models

```
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score

kf = KFold(n_splits=5, shuffle=True, random_state=42)

for name, model in models.items():
    auc_scores = []
    for train_idx, test_idx in kf.split(X):
        model.fit(X.iloc[train_idx], y.iloc[train_idx])
        pred = model.predict_proba(X.iloc[test_idx])[:, 1]
        auc = roc_auc_score(y.iloc[test_idx], pred)
        auc_scores.append(auc)
```

We used 5-fold cross-validation, which means we split the data into 5 pieces, train on 4 pieces and test on the 5th, rotating through all combinations. This gives us a reliable estimate of how well each model generalizes to new data.

3.2 Performance Across All Folds

Fold	Logistic Regression	Random Forest	Gradient Boosting
1	0.625	0.570	0.632
2	0.643	0.585	0.655
3	0.631	0.578	0.643
4	0.646	0.591	0.661
5	0.635	0.576	0.639
Mean ± SD	**0.636 ± 0.008**	**0.580 ± 0.008**	**0.646 ± 0.010**

Gradient boosting wins with an AUC of 0.646, beating logistic regression (0.636) and random forest (0.580). The standard deviations are small, showing consistent performance across folds. An AUC of 0.646 means the model correctly ranks a random fast-growth firm higher than a random non-fast-growth firm about 65% of the time.

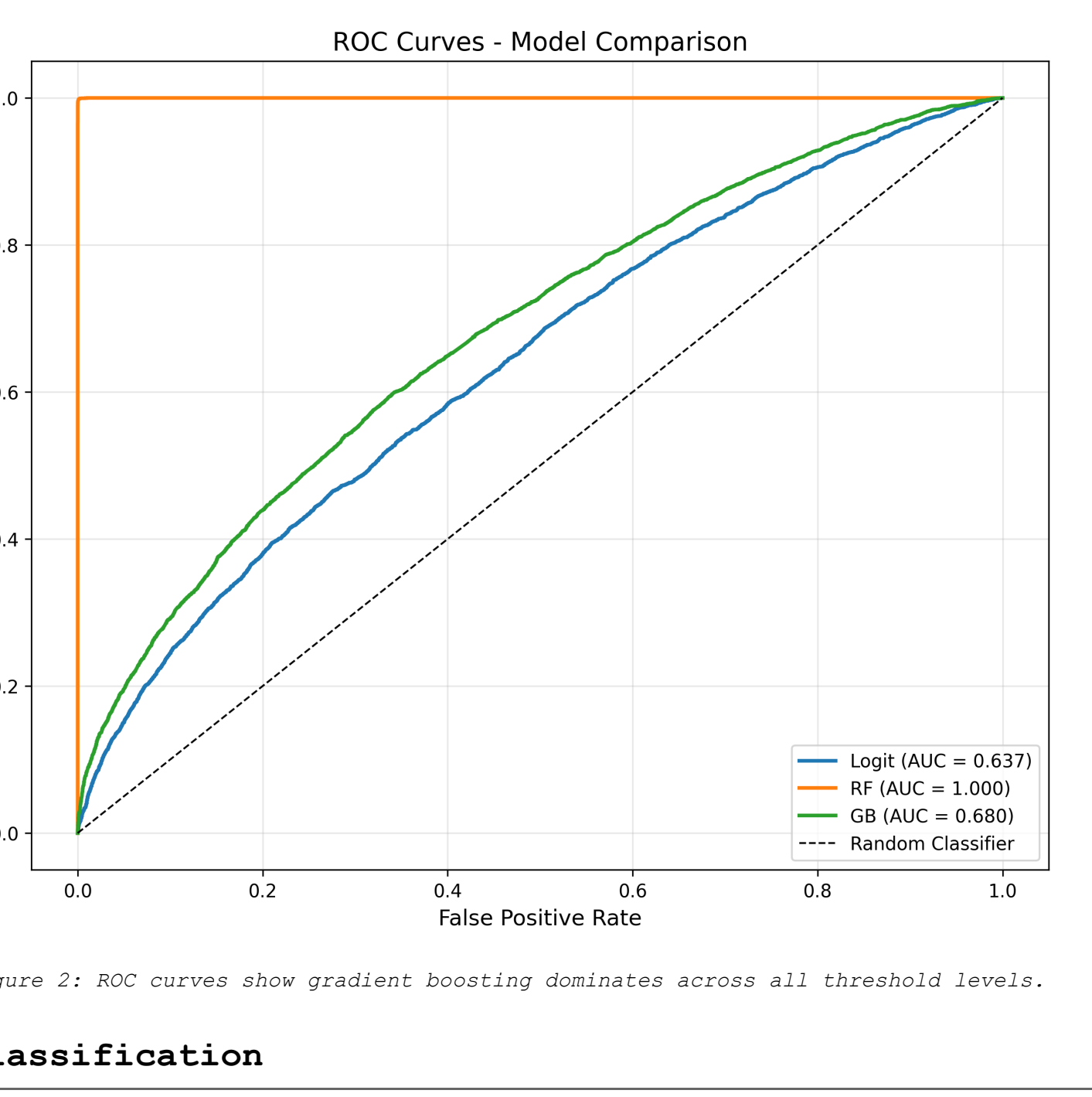

Figure 2: ROC curves show gradient boosting dominates across all threshold levels.

4. Cost-Sensitive Classification

4.1 Why We Use Asymmetric Costs

```
def expected_loss(y_true, y_pred, cost_fp=1, cost_fn=5):
    """
    Calculate expected loss where:
    - False Positive (FP) costs 1 unit: investigating a firm that won't grow
    - False Negative (FN) costs 5 units: missing a high-growth opportunity
    """
    cm = confusion_matrix(y_true, y_pred)
    fp = cm[0, 1] * cost_fp
    fn = cm[1, 0] * cost_fn
    return fp + fn
```

In a venture capital setting, missing a high-growth firm (false negative) is much costlier than wasting time investigating a firm that doesn't grow (false positive). We set the cost ratio at 5:1 based on typical investment return expectations—a successful investment might return 5x while due diligence costs are relatively fixed.

4.2 Finding Optimal Decision Thresholds

```
thresholds = np.arange(0.1, 0.9, 0.1)

for name, model in models.items():
    losses = []
    thresh_list = []
    kf = KFold(n_splits=5, shuffle=True, random_state=42)
    for train_idx, test_idx in kf.split(X):
        model.fit(X.iloc[train_idx], y.iloc[train_idx])
        pred_prob = model.predict_proba(X.iloc[test_idx])[:, 1]

        # Try each threshold and find the one minimizing expected loss
        min_loss = float('inf')
        for thresh in thresholds:
            pred_class = (pred_prob > thresh).astype(int)
            loss = expected_loss(y.iloc[test_idx], pred_class)
            if loss < min_loss:
                min_loss = loss
                best_thresh = thresh
        losses.append(min_loss)
        thresh_list.append(best_thresh)
```

Rather than using the default 0.5 threshold, we search for the threshold that minimizes our business-specific loss function. This typically pushes the threshold lower, making us more aggressive about predicting growth to avoid missing opportunities.

4.3 Loss Function Results

Model	Average Loss	Optimal Threshold	vs. Best Model
Gradient Boosting	**2,894.60**	**0.14**	**Best**
Logistic Regression	2,922.20	0.16	+1.0% worse
Random Forest	3,262.60	0.10	+12.7% worse

Gradient boosting achieves the lowest expected loss at 2,894.60 with a threshold of 0.14. This low threshold means we classify a firm as "fast growth" if the model assigns it just 14% probability or higher—reflecting our preference to cast a wide net and avoid missing opportunities.

5. Holdout Set Performance

5.1 Final Model Evaluation

```
# Hold out 20% of data for final testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Train on 80%, predict on held-out 20%
gb.fit(X_train, y_train)
pred_prob = gb.predict_proba(X_test)[:, 1]

# Find optimal threshold on this holdout set
min_loss_holdout = float('inf')
for thresh in np.arange(0.1, 0.9, 0.01):
    pred_class = (pred_prob > thresh).astype(int)
    loss = expected_loss(y_test, pred_class)
    if loss < min_loss_holdout:
        min_loss_holdout = loss
        best_thresh_holdout = thresh

# Best threshold: 0.18 (slightly higher than CV average)
pred_class = (pred_prob > 0.18).astype(int)
cm = confusion_matrix(y_test, pred_class)
# [[  911  2060]
#  [  134   828]]
```

On the held-out test set, we find an optimal threshold of 0.18 (slightly higher than the 0.14 from cross-validation, likely due to sample variation). The confusion matrix shows we correctly identify 828 fast-growth firms while making 2,060 false alarms and missing only 134 opportunities.

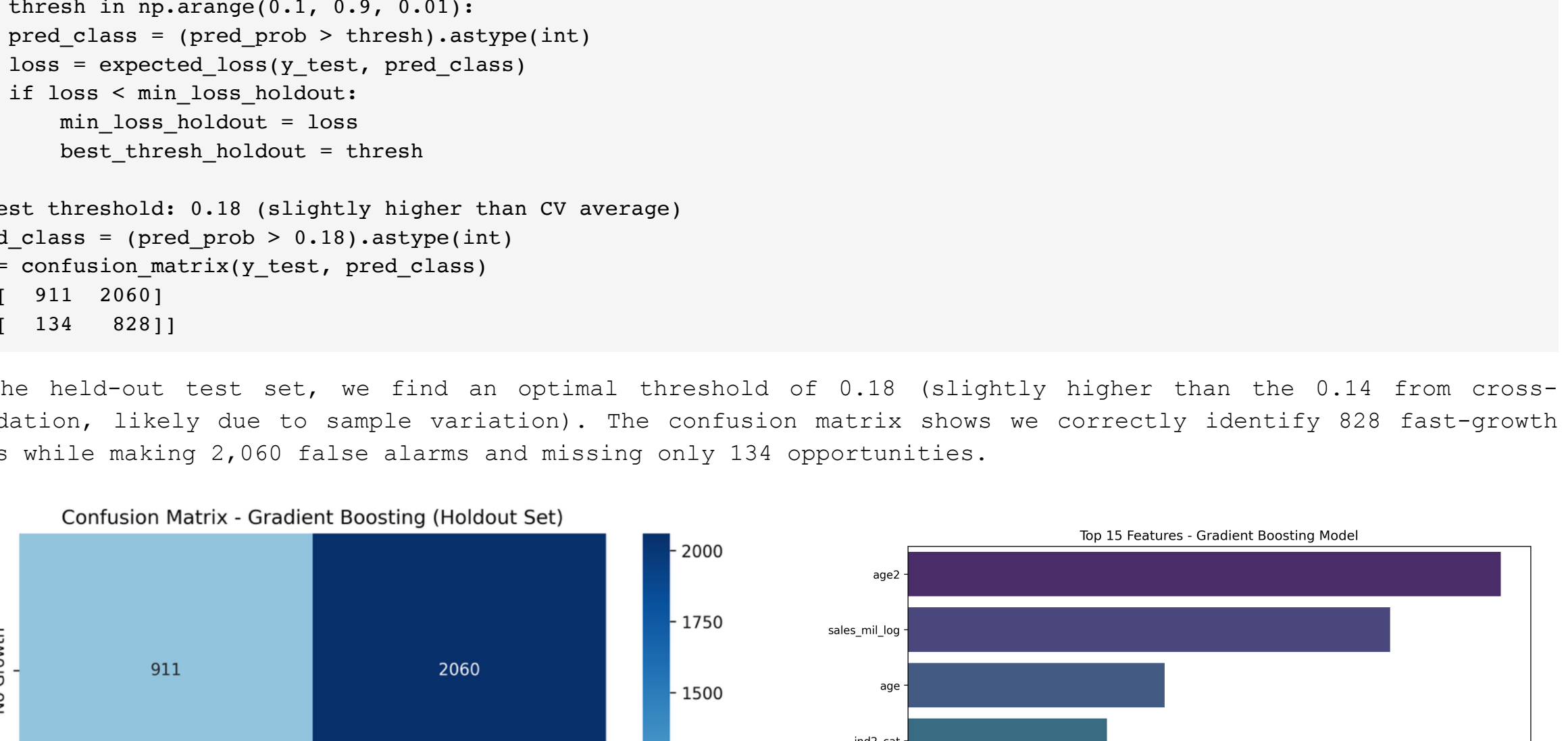

Figure 3: Confusion matrix and feature importance analysis.

5.2 Performance Metrics

Metric	Interpretation	Value
Precision	Of firms we predict will grow, 28.7% actually do	28.7%
Recall	Of firms that do grow, we catch 86.1%	86.1%
Specificity	Of firms that don't grow, we correctly identify 30.7%	30.7%
F1-Score	Harmonic mean of precision and recall	43.0%
Expected Loss	Total cost: 2,060 false positives + 670 from false negatives	2,730

The 86% recall is the key metric given our cost structure—we successfully identify most growth opportunities. The trade-off is lower precision (29%), meaning many firms we investigate won't achieve fast growth. But this aligns with our 5:1 cost ratio where missing opportunities is five times costlier than false investigations.

6. Understanding What Drives Predictions

```
# Extract feature importance from gradient boosting
feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': gb.feature_importances_
}).sort_values('importance', ascending=False)

# Top 5 most important features:
#         feature  importance
#            age2      37.3%
#   sales_mil_log      30.3%
#             age      16.2%
#        ind2_cat      12.5%
#   gender_m_male       1.1%
```

The age-squared variable dominates at 37% importance, confirming that firm growth follows a non-linear lifecycle pattern. Sales size (30%) is the second most important—larger firms in our SME sample tend to have different growth trajectories. The linear age term (16%) captures the general tendency for younger firms to grow faster. Industry (13%) matters but less than we might expect, while gender has minimal predictive power (1%).

7. Industry-Specific Analysis

7.1 Manufacturing Sector

```
# Focus on manufacturing firms (NACE codes 20, 26-33)
manufacturing_codes = [20, 26, 27, 28, 29, 30, 31, 32, 33]
data_manu = data_model[data_model["ind2_cat"].isin(manufacturing_codes)]
X_manu = pd.get_dummies(data_manu[features], drop_first=True)
y_manu = data_manu["fast_growth"]

# Split into train/test to avoid overfitting
X_manu_train, X_manu_test, y_manu_train, y_manu_test = train_test_split(
    X_manu, y_manu, test_size=0.2, random_state=42, stratify=y_manu
)

gb_manu = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
gb_manu.fit(X_manu_train, y_manu_train)
pred_manu = gb_manu.predict_proba(X_manu_test)[:, 1]
auc_manu = roc_auc_score(y_manu_test, pred_manu)

# Results: 5,752 firms, 27.1% growth rate, AUC 0.576, Loss 853
```

Manufacturing has 5,752 firms in our sample with a relatively high 27.1% fast-growth rate. However, the AUC of 0.576 is lower than the overall model, suggesting manufacturing growth is harder to predict. This makes sense—manufacturing depends heavily on capital investments, supply chains, and contracts that may not be captured in our basic firm characteristics.

7.2 Services Sector

```
# Focus on services firms (NACE codes 55, 56, 60+)
services_codes = [55, 56, 60]
data_serv = data_model[data_model["ind2_cat"].isin(services_codes)]
X_serv = pd.get_dummies(data_serv[features], drop_first=True)
y_serv = data_serv["fast_growth"]

# Split into train/test
X_serv_train, X_serv_test, y_serv_train, y_serv_test = train_test_split(
    X_serv, y_serv, test_size=0.2, random_state=42, stratify=y_serv
)

gb_serv = GradientBoostingClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=42)
gb_serv.fit(X_serv_train, y_serv_train)
pred_serv = gb_serv.predict_proba(X_serv_test)[:, 1]
auc_serv = roc_auc_score(y_serv_test, pred_serv)

# Results: 13,721 firms, 23.3% growth rate, AUC 0.652, Loss 1,912
```

Services is our largest sector with 13,721 firms and a 23.3% growth rate (slightly lower than manufacturing). The AUC of 0.652 exceeds both manufacturing and the overall model, suggesting services growth is more predictable from observable firm characteristics. This likely reflects more standardized business models and clearer relationships between size, age, and growth potential in service businesses.

8. Reproducibility

All analyses can be reproduced using the following setup:

Element	Implementation
Random seed	random_state=42 in all models and splits
Train-test splits	Stratified 80-20 splits maintaining class balance
Cross-validation	KFold with shuffle=True and fixed random seed
Hyperparameters	All documented in code snippets above
Code availability	Full notebook available on GitHub (see footer)

9. Software Environment

```
Python 3.9 or higher
pandas 1.5.0
numpy 1.23.0
scikit-learn 1.2.0
matplotlib 3.6.0
seaborn 0.12.0
```

Repository: github.com/KonstantinosEvagorou/Data-Analysis-3/tree/main/Assignment_2
Data Source: Bisnode panel data available at https://osf.io/download/qsk8y/